

Apéndices técnicos de CREApp

Auditoría de ramas, contratos, riesgos y evidencia de validación

Código del Documento:	CREAPP-OPS-02
Versión:	1.0
Autor:	Juan Martín Morales
Repositorio:	https://github.com/juanm4morales/cre-app
Clasificación:	Documentación Técnica de Operaciones
Destinatarios:	Administradores de Sistemas, Infraestructura y DevOps
Fecha de Revisión:	30 de septiembre de 2026

Universidad Nacional de Cuyo — Sistema de Gestión Curricular y Equivalencias (CREApp)

Este documento reúne el detalle de respaldo de los manuales de despliegue y desarrollo.

Cobertura documental: servidores propios y Azure. Este documento no verifica infraestructura ni despliegues en ejecución.

Contenido

Propósito y alcance	1
1. Ramas y diferencias reales	1
2. Restricciones de la receta self-hosted	1
3. Seguridad y entorno	2
4. Migraciones con historial	2
5. Deuda técnica conocida	2
6. Contratos de serialización	3
7. Variables de entorno	3
8. Configuración efectiva	4
9. Referencia de modelos	4
10. Detalle de endpoints	6
11. Importadores y scripts	6
12. Configuración del cliente	7
13. Nginx y despliegue split-origin	7
14. Evidencia de las pruebas	7
15. Diagramas	8
16. Documentos relacionados	8

Propósito y alcance

Versión: 1.0

Detalle de respaldo para los [manuales de despliegue](#) y [desarrollo](#). Aquí se registran las observaciones de auditoría, las diferencias entre ramas y los riesgos conocidos. Ninguna de estas afirmaciones describe el estado en vivo de Azure.

1. Ramas y diferencias reales

La intención de diseño es que **azure** sea la fuente de producto y que **azure-same-origin** sea una variante de despliegue derivada, con diferencias concentradas en routing y despliegue. En las instantáneas auditadas esa intención no se cumple: la variante también modifica el contrato CSRF y los workflows.

Snapshot observado	Rutas y estáticos	CSRF	Workflows de App Service
origin/azure	/api/static/, admin /api/sadmin-creapp-panel/, build con VITE_API_URL=/api y VITE_STATIC_BASE=/api/static/	Devuelve csrfToken en JSON además de cookie; el cliente acepta ambos	deploy.yml y main_cre-app-api.yml filtran main; deploy.yml también azure
origin/azure-same-origin	/static/, admin /sadmin-creapp-panel/, base de Vite fija en /static/	Solo cookie; el cliente lee document.cookie	Ambos workflows filtran main y azure
chore/app-service-same-origin	/api/static/, admin /api/sadmin-creapp-panel/	Devuelve csrfToken en JSON	main_cre-app-api.yml filtra solo main

Consecuencias prácticas:

- La variante con rutas raíz no es un caso donde cambiar VITE_STATIC_BASE alcance: su base está fija en el código. Si se usa esa variante, el **alias** de estáticos y el proxy del panel de administración deben coincidir con ella, o el fallback de la SPA responderá HTML donde se espera el admin y puede producir un bucle de redirección.
- El token CSRF en JSON existe para el caso split-origin, donde la SPA no puede leer la cookie del backend. Con solo cookie, ese contrato no debe darse por compatible.
- Que dos workflows declaren la misma aplicación no prueba qué se despliega: las definiciones no determinan los workflows habilitados ni el artefacto publicado.
- El workflow de Static Web Apps tiene el **push** comentado y conserva los eventos de pull request. No es un despliegue por push.

Las ramas **main**, **dev**, **unidades**, **frontend-modernization** y **legacy/cross-site** están fusionadas y son de referencia. No son objetivos de compatibilidad ni rutas de despliegue.

2. Restricciones de la receta self-hosted

El ejemplo de Nginx y el build con **/api/static/** forman un contrato: **STATIC_URL=/api/static/**, panel en **/api/sadmin-creapp-panel/**, **VITE_API_URL=/api** y **VITE_STATIC_BASE=/api/static/**. Antes de adaptar la receta, revise las rutas reales de la versión que va a instalar.

El orden importa: el frontend se compila antes de **collectstatic** porque su salida forma parte de los estáticos de Django, y **current** se publica solo después de compilar, migrar y recolectar. Publicar antes de tiempo hace que Nginx sirva archivos nuevos mientras Gunicorn ejecuta el release anterior.

Sobre cuentas y permisos: **creapp-build** prepara el release, **creapp** ejecuta la aplicación y solo escribe en **backend/staticfiles**, y **www-data** lee código y estáticos. **/etc/creapp/creapp.env** es **root:creapp** con modo

0640, de modo que el grupo del servidor web no puede leer los secretos. Estas relaciones se verificaron en un contenedor descartable, no en una instalación real.

`systemd-run` se usa aquí por conveniencia para pasar `EnvironmentFile` sin leer secretos en una shell. Si la política del sistema no permite unidades transitorias, defina una unidad `oneshot` equivalente. Los avisos `Underfull` de un archivo no habilitado no son fallos del procedimiento.

3. Seguridad y entorno

HSTS. Está desactivado por defecto. Un `check --deploy` aislado con `SECURE_HSTS_SECONDS=0` informó W004; con 3600 y las subopciones en falso, informó W005 y W021. Son advertencias de configuración: habilite HSTS solo después de confirmar HTTPS, el dominio y todos los subdominios, y hágalo de forma gradual. No active `includeSubDomains` ni `preload` únicamente para silenciar la salida.

TLS hacia PostgreSQL. `settings.py` usa parámetros separados y no interpreta `POSTGRES_SSLMODE`. Para una base remota configure `PGSSLMODE` y `PGSSLROOTCERT` en el entorno del proceso, que `libpq` sí reconoce, y verifique la validación del certificado.

Proxy de acceso. `ProxyAccessMiddleware` solo se activa si se define `PROXY_ACCESS_SECRET`; entonces exige la cabecera `X-Proxy-Access-Secret` salvo en las rutas exentas, que por defecto incluyen `/healthz`, el panel y los estáticos. El cliente usa cookie en lugar de cabecera. En el repositorio existe además `frontend/middleware.js`, que inyecta esa cabecera con `@vercel/functions`; es una función de edge y no implica que un runtime de Vercel esté configurado. Solo actívelo con un proxy de confianza diseñado para ello.

Dependencias frontend. `npm ci` informó 16 avisos: 1 bajo, 4 medios y 11 altos. No se aplicó `npm audit fix` automáticamente. Revíselos por separado: una actualización automática de dependencias cambia el comportamiento de compilación y debe probarse.

4. Migraciones con historial

`0011_seed_ip_tipo_actividad` crea los nombres de tipo IP que faltan. En una versión anterior de ese archivo, un nombre existente podía ver su dedicación reescrita a IP. La corrección publicada en la rama de producto impide que la semilla cambie registros existentes; el importador ahora rechaza nombres que ya tengan otra dedicación. Ninguna de las dos cosas repara una base que ya ejecutó la migración anterior: el efecto histórico requiere una decisión explícita sobre conservar o reiniciar los datos, no una migración automática.

0012 agrega cuatro campos de texto a `Programa` y no crea relaciones de competencias. El nombre de la migración puede sugerir lo contrario.

El plan de liberación de Azure y el resguardo de datos anteriores a la migración 0009 se conservan en `../AZURE_RELEASE_PLAN.md` y `../LEGACY_MODALIDAD_MIGRATION.md`. Ambos registran observaciones de una auditoría previa, no un estado verificado en el momento de leerlos.

5. Deuda técnica conocida

- **ClaseCalendarioSerializer no valida el ámbito docente.** Acepta `programa` y `dia_clase` sin comprobar la asignación, tanto al crear como al actualizar. El `queryset` filtra la lectura, pero la escritura puede apuntar a otro programa. La acción `generar-rango` sí valida.
- **Solapamiento de asignaciones.** La detección de períodos coincidentes es una validación de Python que lee y luego escribe. Dos peticiones concurrentes pueden pasar el chequeo. Faltan una restricción de exclusión o un bloqueo transaccional, y pruebas de concurrencia.
- **Endpoints temporales de docente.** `temporal/espacio`, `temporal/carga-horaria` y `temporal/asignarme` crean y modifican catálogo, relación curricular y asignaciones reales. Son una prueba funcional y deben retirarse cuando exista la integración institucional. Exigen perfil `DOCENTE`, no solo rol activo, por lo que un usuario `staff` con perfil docente también puede usarlos.
- **Créditos en la vista.** La pantalla de espacios calcula con 27 horas por crédito cuando aún no recibió la configuración, mientras el backend usa el valor configurado y 25 por omisión. La vista puede mostrar un número distinto al que cálculo el servidor.

- **Catálogo de tipos.** La distinción IP/TA en `TipoActividad.tipo_dedicacion` condiciona serializers, listas, formularios y calendario. Cambiarla exige unificar esos puntos.
- **Credenciales de docentes importados.** `import_academic_xlsx` deja contraseñas inutilizables; hace falta un flujo administrativo para establecerlas.
- **Pruebas con nombre desactualizado.** `PasswordValidationGapTest` en `accounts/tests.py` describe una brecha que ya no existe: ambos serializers validan la contraseña. Corrija el nombre y los comentarios al tocar esa suite.
- **Inventario de pruebas del backend.** Además de `accounts`, `academics` y `planning`, existen `config/tests_csrf.py` y `config/tests_proxy_access.py`. El segundo prueba rutas históricas del panel.
- **Documentación de API desactualizada.** `docs/API.md` usa barra final, recursos inexistentes y filtros no implementados. El router es la referencia.
- **Integración SIU no implementada.** Los documentos que describen un contrato con SIU Guaraní son una solicitud; no existe cliente ni endpoints en el backend. El flujo comprobable es la importación de XLSX.

6. Contratos de serialización

No todos los atributos del modelo están expuestos. Antes de asumir que un campo existe en la API, revise la lista `fields` del serializer.

- Las claves foráneas se devuelven como identificadores enteros, no como objetos anidados.
- **Programa** acepta plan-espacio, año y textos al crear; al actualizar solo el año y los textos, de modo que no se mueve de plan-espacio.
- **Unidad** acepta programa, número y descripción; al actualizar solo número y descripción. Las competencias se asignan por la acción dedicada, que reemplaza el conjunto completo y conserva el orden recibido.
- **Actividad** exige unidades activas del mismo programa. Para interacción pedagógica se exige clase del mismo programa y que el día esté habilitado si hay reglas; no admite fechas de trabajo autónomo ni horas por encima de la franja del día. Para trabajo autónomo se prohíbe la clase de calendario y se valida el orden de las fechas. Al actualizar, `unidad_ids` es opcional y si se omite conserva la relación.
- **DiaClasePrograma** exige que la hora de inicio y la de fin estén ambas presentes o ambas vacías.
- **AsignacionDocente** rechaza usuarios inactivos o que no son docentes, fechas invertidas y períodos solapados para el mismo docente y espacio. El campo `activo` es un valor calculado por fecha, no una columna.
- **ActividadAjuste** exige horas extra por encima de cero cuando el tipo es extensión, y que la clase destino pertenezca al programa de la actividad.
- Las horas se almacenan como decimales. Las pantallas capturan minutos y convierten antes de enviar.

7. Variables de entorno

El backend carga `backend/.env` con `load_dotenv` desde `BASE_DIR`. Las listas se separan por comas. Los booleanos aceptan `true`, `1`, `yes` y `on`; cualquier otro valor se interpreta como falso.

Variable	Por omisión	Uso
<code>DEBUG</code>	<code>True</code>	Debe ser <code>False</code> en producción
<code>SECRET_KEY</code>	Clave insegura fija si <code>DEBUG</code> es verdadero	Sin ella y con <code>DEBUG=False</code> Django no arranca
<code>ALLOWED_HOSTS</code>	<code>localhost,127.0.0.1</code>	Hostnames exactos, sin esquema
<code>POSTGRES_DB</code> , <code>POSTGRES_USER</code> , <code>POSTGRES_PASSWORD</code>	Sin valor útil	Base y credenciales
<code>POSTGRES_HOST</code>	<code>localhost</code>	Host o FQDN
<code>POSTGRES_PORT</code>	<code>5432</code>	Puerto
<code>CORS_ALLOWED_ORIGINS</code>	<code>http://localhost:5173,http://127.0.0.1:5173</code>	Orígenes exactos con esquema

CSRF_TRUSTED_ORIGINS	Igual que CORS	Orígenes de peticiones que modifican datos
CORS_ALLOW_CREDENTIALS	Siempre True	No se lee del entorno
CSRF_COOKIE_SECURE, SESSION_COOKIE_SECURE	False	True con HTTPS
CSRF_COOKIE_SAMESITE, SESSION_COOKIE_SAMESITE	Lax	None exige Secure
SECURE_SSL_REDIRECT	False	True con proxy HTTPS correcto
SECURE_HSTS_SECONDS	0	Habilitar gradualmente
SECURE_HSTS_INCLUDE_SUBDOMAINS, SECURE_HSTS_PRELOAD	False	Requieren decisión de dominio
SECURE_CONTENT_TYPE_NOSNIFF	True	Mantener
X_FRAME_OPTIONS	DENY	Cambiar solo con requisito
PROXY_ACCESS_SECRET	Vacío, desactiva el middleware	Secreto compartido con el proxy de acceso
PROXY_ACCESS_EXEMPT_PATHS	/healthz,/sadmin-creapp-panel/,/api/sadmin-creapp-panel/,/api/static/	Rutas sin exigir la cabecera del proxy

Variables de compilación del frontend: `VITE_API_URL` (base del cliente HTTP, por omisión `/api`) y `VITE_STATIC_BASE` (base de Vite, por omisión `/`). Se incrustan en el JavaScript público: nunca pueden contener secretos.

8. Configuración efectiva

- Idioma `es-ar`, zona horaria `America/Argentina/Mendoza`, `USE_TZ=True`.
- `DEFAULT_CRE_HOURS = 25` alimenta `ConfiguracionCRE.horas_por_cre`.
- `STATIC_URL = /api/static/`, `STATIC_ROOT = backend/staticfiles` y `STATICFILES_DIRS` apunta a `frontend/dist`, que además es el directorio de plantillas. El build del frontend entra en los estáticos, por eso debe compilarse antes de `collectstatic`.
- `WhiteNoise` con almacenamiento de estáticos comprimidos y `manifest`.
- `DRF`: autenticación por sesión, permiso `IsAuthenticated` por defecto y paginación por número de página con 100 registros.
- Rutas de redirección de login y logout apuntan al panel en `/api/sadmin-creapp-panel/`.
- Orden de middleware: seguridad, acceso del proxy, `WhiteNoise`, sesión, CORS, común, CSRF, autenticación, mensajes y protección de clics.
- `/healthz` responde texto plano `ok` y no consulta la base de datos.
- El orden de URLs es: salud, panel, estáticos, API y, al final, una captura que devuelve la SPA. Esa última ruta hace que un path mal escrito responda el HTML de la aplicación con estado 200 en lugar de un 404.

9. Referencia de modelos

academics

Modelo	Campos y reglas
--------	-----------------

ConfiguracionCRE	horas_por_cre (por omisión 25), actualizado_en automático. Singleton por convención: <code>save()</code> reutiliza la primera clave existente, <code>get_instance()</code> usa la clave 1 y <code>get_hours_per_cre()</code> cae al valor por omisión. Sin restricción de unicidad en la base
UnidadAcademica	nombre, sigla única (máx. 10)
Carrera	nombre, codigo único (máx. 20), unidad_academica con PROTECT, nivel PG/G por omisión G
PlanEstudio	carrera con PROTECT, nombre, ordenanza única, descripcion opcional, credits mínimo 1, vigente_desde, vigente_hasta opcional. Único por carrera y nombre. horas es una propiedad: créditos por horas por CRE. La relación con espacios es M2M a través de PlanEstudioEC
PlanEstudioEC	Une plan y espacio, ambos con PROTECT, único por par. No es solo metadato: Programa.plan_estudio_ec usa CASCADE
EspacioCurricular	codigo único e indexado (máx. 20), nombre indexado, tipo_espacio T1-T4, anio_cursada 1 a 10, periodo ANUAL/1S/2S, credits mínimo 1, horas_ip y horas_ta no negativas. horas_totales es propiedad
Competencia	plan_estudio con PROTECT, codigo (máx. 50) único por plan, nombre, descripcion opcional, activo indexado. Orden por plan y código

planning

Modelo	Campos y reglas
AsignacionDocente	docente y espacio_curricular con PROTECT, categoria TIT/ADJ/ASO/JTP/AY1/AY2, vigente_desde, vigente_hasta opcional. Índices por docente y espacio con fecha. Restricción en base: <code>vigente_desde <= vigente_hasta</code> . El manager <code>activas(fecha)</code> usa hoy por omisión y límites inclusivos. <code>clean()</code> rechaza solapamientos del mismo docente y espacio, pero <code>save()</code> no lo invoca. <code>activo</code> y <code>esta_vigente(fecha)</code> son propiedades calculadas
Programa	plan_estudio_ec con CASCADE, anio_academico mínimo 1939 e indexado, descripcion, cuatro textos de 0012 (fundamentacion, objetivos_generales, objetivos_especificos, competencias) y activo. Único por plan-espacio y año
Unidad	programa con CASCADE, numero mínimo 1, descripcion, activo. Único por programa y número
UnidadCompetencia	unidad con CASCADE, competencia con PROTECT, orden mínimo 1. Único por par. <code>clean()</code> exige que la competencia pertenezca al plan del programa de la unidad, y <code>save()</code> sí invoca <code>full_clean()</code>
DiaClasePrograma	programa con CASCADE, dia_semana 0 lunes a 6 domingo, hora_inicio y hora_fin opcionales, activo. Único por programa, día y franja. Restricción condicional: inicio antes que fin cuando ambos existen
ClaseCalendario	programa con CASCADE, dia_clase opcional que se anula si se borra, fecha indexada, estado PLAN/DICT/CANC, observaciones. Único por programa y fecha. <code>clean()</code> exige que el día pertenezca al programa y <code>save()</code> invoca <code>full_clean()</code>
TipoActividad	nombre único, descripcion opcional, tipo_dedicacion IP/TA por omisión TA. <code>get_tipo_actividad_otros()</code> crea "Otros" si falta
Actividad	programa con CASCADE, M2M unidades, tipo_actividad con SET_DEFAULT a "Otros", descripcion, modalidad_trabajo IND/EQU, horas decimal no negativo, activo, clase_calendario opcional y fechas de trabajo autónomo. <code>clean()</code> valida el orden de fechas y que la clase pertenezca al programa, pero <code>save()</code> no lo invoca. <code>espacio_curricular</code> es propiedad derivada del programa

ActividadAjuste	actividad con CASCADE, tipo REC/EXT, motivo, horas_ip_extra no negativo, fecha_evento opcional, clase_destino opcional, creado_por con PROTECT, creado_en automático. clean() exige horas extra por encima de cero en una extensión y que la clase destino pertenezca al programa; save() invoca full_clean()
-----------------	---

10. Detalle de endpoints

Endpoint	Comportamiento
/api/unidades-academicas	Orden por sigla; escritura solo con rol admin
/api/carreras	Filtro unidad_academica_id; orden por nombre
/api/configuracion-cre	Orden por fecha de actualización descendente
/api/planes-estudio	Filtro carrera_id; serializer valida que la fecha de fin no sea anterior a la de inicio
/api/planes-estudio-ec	Filtros plan_estudio_id y espacio_curricular_id; el borrado se rechaza si hay programas
/api/espacios-curriculares	Orden por nombre
/api/competencias	Ocultas las inactivas salvo include_inactive=1; filtro plan_estudio_id; el borrado es lógico
/api/usuarios	Solo docentes que no son superusuarios; orden por nombre de usuario; el borrado desactiva
/api/asignaciones-docentes	Escritura solo admin; filtro por docente y espacio
/api/tipos-actividad	Orden por nombre; escritura solo admin
/api/programas	Filtro plan_estudio_ec_id; ámbito por asignación; el borrado desactiva el programa y sus unidades y actividades
/api/unidades	Filtros programa_id y plan_estudio_ec_id; el detalle incluye las competencias asociadas
/api/dias-clase	Filtros por programa y plan-espacio; el borrado es lógico
/api/actividades	Filtros por programa y plan-espacio; el detalle añade identificadores de unidad e indicadores de tipo y ajustes
/api/clases-calendario	Filtros por programa, plan-espacio y rango de fechas; orden por fecha
/api/espacios-asignados	Admin ve todos los espacios; docente, los asignados hoy. Devuelve un array simple, sin paginación

Los listados usan el serializer de lectura; para crear y actualizar se usan serializers distintos, con validaciones que dependen de la acción. Los borrados lógicos responden con éxito sin eliminar la fila.

11. Importadores y scripts

import_academic_xlsx acepta la ruta del archivo y exige --unidad-sigla, --unidad-nombre, --plan-credits y --vigente-desde; --career-level y --dry-run son opcionales. Lee la hoja obligatoria Propuestas, con las columnas id_propuesta, nombre, nombre_plan, titulo y normativa, y las hojas opcionales Espacios, Competencias y Docentes. La hoja Areas se ignora. Trabaja en una transacción y el modo de ensayo revierte al final. Inserta o actualiza por sigla de unidad, código de carrera, ordenanza de plan, código de espacio, par plan-espacio, par plan y código de competencia, e identidad o correo del docente. El estado de la propuesta infiere la fecha de fin cuando indica cierre o baja. Los docentes nuevos se identifican como docente-<id> cuando hay legajo, reciben contraseña inutilizable y una asignación vigente desde la fecha indicada; también emite advertencias cuando faltan hojas o un docente referencia un espacio inexistente.

import_tipo_actividad_xlsx acepta la ruta y el tipo de dedicación, que por omisión es trabajo autónomo, además del modo de ensayo. Lee la primera hoja desde la fila 3 y toma el nombre, la descripción, los ejemplos y

las notas de columnas específicas para componer la descripción. Inserta o actualiza por nombre y **falla con un error claro cuando la dedicación importada difiere de la existente**, en vez de sobrescribirla.

`scripts/release.sh` ejecuta las migraciones y, si encuentra el archivo esperado, la importación académica con la sigla FCE. No sirve como comando genérico de inicialización.

`scripts/db-export.sh` y `scripts/db-import.sh` usan las credenciales del archivo de entorno, o el indicado por `ENV_FILE`, con un analizador simple de líneas `NOMBRE=valor`. No interpretan sintaxis de shell ni configuran SSL: para una base remota hay que exportar además las variables de TLS de libpq. El import crea la base si falta y restaura con limpieza de objetos, por lo que puede reemplazar datos.

12. Configuración del cliente

- `QueryClient` global con `staleTime` de cinco minutos, sin recarga al enfocar la ventana y con las herramientas de desarrollo solo en desarrollo.
- El interceptor de axios emite el evento `cre:api-data-changed` tras cada mutación; `useApiAutoRefresh` vuelve a pedir datos con un retardo de 150 ms al recibirlo, al enfocar la ventana y al volver a ser visible el documento.
- La sesión se guarda en `localStorage` bajo `cre_auth_user` solo para presentar la interfaz.
- La selección docente usa `sessionStorage` con las claves del espacio, del plan-espacio y del nombre del espacio.
- El tema se guarda en `localStorage` como `cre_theme` y se aplica como clase en el elemento raíz, que `index.html` inicializa antes de montar React.
- `App.css` importa nueve hojas: layout, componentes, tablas, autenticación, modales, calendario, docente, utilidades y responsive.
- El proxy de desarrollo de Vite reenvía las rutas de API, del panel y de estáticos al backend local, y reescribe el origen de la petición al servidor de desarrollo para que la validación CSRF lo reconozca.
- Vitest usa jsdom y el archivo de configuración de pruebas; no hay configuración de cobertura. Existe configuración de ESLint, pero ningún script de npm la ejecuta.

13. Nginx y despliegue split-origin

Django confía en `X-Forwarded-Proto`, por lo que el proxy debe sobrescribir esa cabecera y las de `Host`, `X-Real-IP`, `X-Forwarded-For` y `X-Forwarded-Host`.

En la variante split-origin con Static Web Apps, el frontend se compila con la URL absoluta del backend en `VITE_API_URL`, y ese valor queda visible en el código del navegador. El backend debe declarar el origen exacto del frontend en CORS y CSRF, sin comodines. Al estar en sitios distintos, las cookies requieren `SameSite=None` y `Secure`, y los navegadores pueden bloquearlas por sus políticas de terceros aunque la configuración sea correcta: hay que probarlas en navegadores reales.

El repositorio incluye `frontend/middleware.js`, una función de edge que inyecta la cabecera de acceso con `@vercel/functions` para las rutas de API, panel y estáticos. Su presencia no implica que exista un runtime de Vercel configurado. El workflow de Static Web Apps tiene el disparo por push desactivado y solo gestiona previsualizaciones de pull request.

14. Evidencia de las pruebas

El recorrido de aceptación se ejecutó en contenedores descartables, contra una copia temporal del código elegido. Pasaron 54 pruebas frontend, 78 pruebas Django, las migraciones desde base vacía, la comprobación de estáticos, una sesión real con CSRF, el panel de administración, y un ciclo de respaldo y restauración con coincidencia del recuento de la tabla usada en la prueba.

Límites: no cubre Azure, una VM con systemd, DNS o certificados públicos, PostgreSQL remoto, firewall institucional ni el comportamiento real de cookies en navegadores. La comparación del respaldo se limitó a una tabla, no a todos los datos. El tiempo de ejecución de la corrida no es una estimación de capacidad.

Dos detalles del propio harness que conviene conocer: el cliente de pruebas de Django usa HTTP local, por lo que solo ese proceso desactiva la redirección a HTTPS, mientras el servicio bajo prueba la mantiene; y un intento

inicial falló con 400 por no incluir el host de prueba en `ALLOWED_HOSTS`, un defecto del fixture que no cambió configuración del producto.

El harness es reproducible y está en [validation/](#). Ejecuta pruebas, no despliega.

15. Diagramas

Las fuentes editables están en [diagrams/](#), en formato TikZ con sus PDF y SVG.

- Catálogo académico y planificación: `classes-academics`, `classes-planning`, `classes-calendar`, `classes-identity`. No incluyen `ConfiguracionCRE`, que se documenta en la tabla del manual de desarrollo.
- Topología del servidor propio: `deployment`.
- Rutas HTTP: `http-routing`.
- Flujos de despliegue: `deployment-workflows`.

16. Documentos relacionados

- [DEPLOYMENT_MANUAL.md](#): procedimiento de despliegue.
- [DEVELOPER_MANUAL.md](#): arquitectura y contratos internos.
- [../AZURE_RELEASE_PLAN.md](#): controles de liberación en Azure.
- [../LEGACY_MODALIDAD_MIGRATION.md](#): resguardo de datos anteriores a la migración 0009.